

Instrukcja wdrożenia predyktora prądów do ROS2

Projekt: PUTM_EV_FF_CurrentLoop — Power Limiter / Torque Vectoring

Data: 2026-07-01

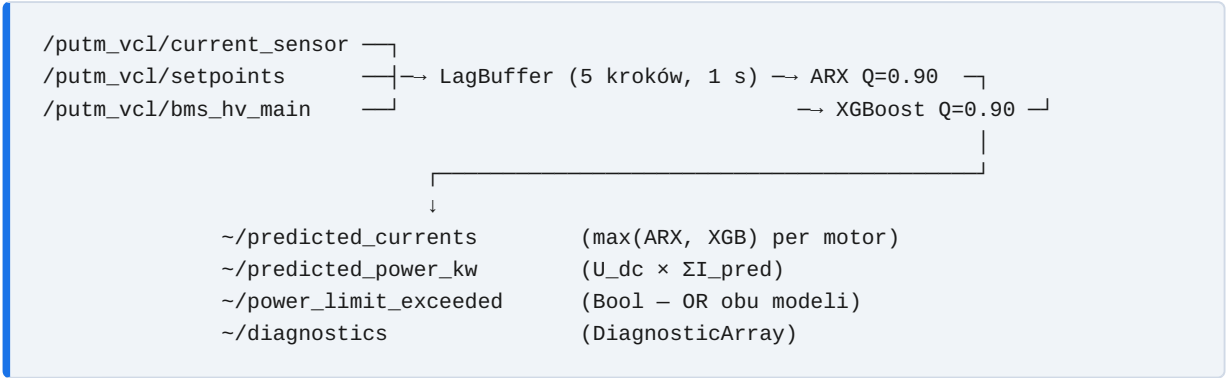
Autor: Michał Błotniak

Przegląd architektury

Pakiet `current_predictor` zawiera dwa modele predykcji prądu działające równolegle jako konserwatywny ensemble:

Model	FN	Recall	ms/próbkę	Rola
ARX Q=0.90	1	99.9%	0.001	Główny predyktor (liniowy, trivialny port do C)
XGBoost Q=0.90	0	100%	0.053	Drugi głos bezpieczeństwa (nieliniowy)

Logika consensus: flaga `power_limit_exceeded = True` jeśli **którykolwiek** z modeli przewiduje moc > 80 kW. Predykcja prądu to element-wise maximum obu modeli (najbardziej konserwatywna wartość per silnik).



Wymagania

ROS2

Kompatybilne dystrybucje: **Humble, Iron, Jazzy**. Wymagane pakiety:

```
sudo apt install ros-$ROS_DISTRO-diagnostic-msgs
```

Python (środowisko ROS2)

```
pip install xgboost numpy
```

Pakiety muszą być dostępne w środowisku Pythona używanym przez ROS2 (nie w venv). Sprawdź:

```
python3 -c "import xgboost; print(xgboost.__version__)"
```

 po sourcing'u ROS2.

Struktura plików projektu

```
PUTM_EV_FF_CurrentLoop/
├─ src/
│   ├── export_models.py      ← skrypt eksportu wag
│   └─ functions/
│       ├── arx.py
│       └─ ...
├─ data/model/training_data.csv
├─ model_weights/             ← generowany przez export_models.py
└─ ros2_pkg/
    └─ current_predictor/     ← pakiet do wdrożenia
```

Krok 1 — Eksport wag modeli

Przejdź do katalogu projektu i uruchom skrypt eksportu. Skrypt trenuje oba modele na pełnym zbiorze treningowym i zapisuje wagi do katalogu `model_weights/`.

```
cd /ścieżka/do/PUTM_EV_FF_CurrentLoop
python src/export_models.py
```

Oczekiwany wynik:

```

=====
Model export: ARX Q=0.90 + XGBoost Q=0.90
=====

Loading training data...
  Train runs [2, 3, 4, 5, 6, 7, 8]: 3730 samples, 31 features

[1/2] Training ARX Q=0.90 ...
  Done in ~3200 ms
  Saved -> model_weights/arx_q90_weights.json

[2/2] Training XGBoost Q=0.90 (4 motors) ...
  Motor FL -> model_weights/xgb_q90_FL.ubj
  Motor FR -> model_weights/xgb_q90_FR.ubj
  Motor RL -> model_weights/xgb_q90_RL.ubj
  Motor RR -> model_weights/xgb_q90_RR.ubj
  Total: ~2.0 s

```

Po zakończeniu katalog `model_weights/` zawiera:

Plik	Zawartość
<code>arx_q90_weights.json</code>	Macierz wag ARX (4×31) + intercept (4)
<code>xgb_q90_FL.ubj</code>	Model XGBoost dla silnika FL
<code>xgb_q90_FR.ubj</code>	Model XGBoost dla silnika FR
<code>xgb_q90_RL.ubj</code>	Model XGBoost dla silnika RL
<code>xgb_q90_RR.ubj</code>	Model XGBoost dla silnika RR
<code>xgb_meta.json</code>	Kolejność cech i metadane

Krok 2 — Skopiowanie wag do pakietu ROS2

Wagi muszą znaleźć się wewnątrz pakietu, żeby `colcon install` zainstalował je do `share/`:

```
cp -r model_weights/* ros2_pkg/current_predictor/model_weights/
```

Weryfikacja:

```
ls ros2_pkg/current_predictor/model_weights/
# arx_q90_weights.json  xgb_meta.json
# xgb_q90_FL.ubj      xgb_q90_FR.ubj  xgb_q90_RL.ubj  xgb_q90_RR.ubj
```

Krok 3 — Skopiowanie pakietu do workspace ROS2

```
cp -r ros2_pkg/current_predictor ~/ros2_ws/src/
```

Struktura po skopiowaniu:

```
~/ros2_ws/src/  
└─ current_predictor/  
    ├── package.xml  
    ├── setup.py  
    ├── setup.cfg  
    ├── resource/current_predictor  
    ├── current_predictor/  
    │   ├── __init__.py  
    │   ├── node.py           ← główny node ROS2  
    │   ├── lag_buffer.py     ← bufor przesuwany (5 kroków)  
    │   ├── arx_predictor.py  ← inferencja ARX  
    │   └── xgb_predictor.py  ← inferencja XGBoost  
    ├── model_weights/       ← wagi modeli (skopiowane w Kroku 2)  
    ├── launch/  
    │   └── current_predictor.launch.py  
    └── config/  
        └── params.yaml
```

Krok 4 — Budowanie pakietu

```
cd ~/ros2_ws  
colcon build --packages-select current_predictor  
source install/setup.bash
```

Sprawdzenie poprawności instalacji:

```
ros2 pkg list | grep current_predictor  
# current_predictor  
  
ros2 run current_predictor current_predictor_node --ros-args --help
```

Krok 5 — Podpięcie wiadomości VCL (sekcja ADAPTER)

Node używa `std_msgs/Float32MultiArray` jako placeholderów. Musisz je zastąpić rzeczywistymi typami wiadomości z pakietu `putm_vcl` / `putm_vcl_msgs`.

Otwórz plik `current_predictor/node.py` i znajdź sekcję `ADAPTER CALLBACKS` (ok. linia 80). Są tam trzy metody do zaadaptowania.

_cb_currents — prądy inwertorów

Aktualny topic: `/putm_vcl/current_sensor`

Format CDR: $4 \times \text{uint16}$, offset 4 bajtów (standardowy nagłówek CDR)

```
# PRZED (placeholder):
def _cb_currents(self, msg: Float32MultiArray) -> None:
    self._currents = np.array(msg.data[:4], dtype=np.float64)
    self._has_current_data = True

# PO (z rzeczywistym typem putm_vcl):
from putm_vcl_msgs.msg import CurrentSensor # <-- dodaj import na górze pliku

def _cb_currents(self, msg: CurrentSensor) -> None:
    self._currents = np.array(
        [msg.i_fl, msg.i_fr, msg.i_rl, msg.i_rr],
        dtype=np.float64
    )
    self._has_current_data = True
```

Zmień też typ subskrypcji w `__init__`: `self.create_subscription(CurrentSensor, cur_topic, self._cb_currents, 10)`

_cb_torques — setpointy momentu

Aktualny topic: `/putm_vcl/setpoints`

Format CDR: $4 \times \text{int32}$ [Nm], offset 4 bajtów

```
# PRZED (placeholder):
def _cb_torques(self, msg: Float32MultiArray) -> None:
    self._t_sum = float(sum(msg.data[:4]))

# PO:
from putm_vcl_msgs.msg import Setpoints # <-- dodaj import

def _cb_torques(self, msg: Setpoints) -> None:
    self._t_sum = float(msg.t_fl + msg.t_fr + msg.t_rl + msg.t_rr)
```

_cb_voltage — napięcie szyny DC

Aktualny topic: `/putm_vcl/bms_hv_main`

Format CDR: `uint16 raw`, gdzie napięcie [V] = `raw × 0.1` (np. 5633 → 563.3 V)

```
# PRZED (placeholder):
def _cb_voltage(self, msg: Float32) -> None:
    self._u_dc = float(msg.data)

# PO:
from putm_vcl_msgs.msg import BmsHvMain # <-- dodaj import

def _cb_voltage(self, msg: BmsHvMain) -> None:
    self._u_dc = float(msg.voltage_sum) * 0.1 # raw → V
```

Nazwy pól (`i_fl`, `t_fl`, `voltage_sum`) zależą od definicji `.msg` w Twoim pakiecie `putm_vcl`. Dostosuj je do rzeczywistych nazw.

Po zmianach przebuduj pakiet (`colcon build --packages-select current_predictor`).

Krok 6 — Uruchomienie

Domyślne topici (po zaadaptowaniu do VCL)

```
ros2 launch current_predictor current_predictor.launch.py \
  current_topic:=/putm_vcl/current_sensor \
  torque_topic:=/putm_vcl/setpoints \
  voltage_topic:=/putm_vcl/bms_hv_main
```

Alternatywnie — z parametrami w yaml

Edytuj `config/params.yaml`:

```
current_predictor:
  ros__parameters:
    current_topic: /putm_vcl/current_sensor
    torque_topic: /putm_vcl/setpoints
    voltage_topic: /putm_vcl/bms_hv_main
    publish_rate_hz: 5.0
    power_limit_w: 80000.0
```

Następnie:

```
ros2 launch current_predictor current_predictor.launch.py
```

Komunikat o poprawnym starcie

```
[current_predictor]: Loading models from ../model_weights  
[current_predictor]: Models loaded (ARX Q=0.90 + XGBoost Q=0.90)  
[current_predictor]: CurrentPredictorNode ready at 5 Hz
```

Weryfikacja działania

Sprawdzenie aktywnych topiców

```
ros2 topic list | grep current_predictor  
# /current_predictor/predicted_currents  
# /current_predictor/predicted_currents_arx  
# /current_predictor/predicted_currents_xgb  
# /current_predictor/predicted_power_kw  
# /current_predictor/power_limit_exceeded  
# /current_predictor/diagnostics
```

Podgląd predykcji w czasie rzeczywistym

```
# Predykowane prądy [I_FL, I_FR, I_RL, I_RR]:  
ros2 topic echo /current_predictor/predicted_currents  
  
# Flaga limitera (True = ogranicz moment):  
ros2 topic echo /current_predictor/power_limit_exceeded  
  
# Predykowana moc w kW:  
ros2 topic echo /current_predictor/predicted_power_kw
```

Szczegółowa diagnostyka

```
ros2 topic echo /current_predictor/diagnostics
```

Przykładowe wyjście:

```

status:
- name: current_predictor
  level: 0   # OK (2 = WARN = przekroczono 80 kW)
  message: OK
  values:
    - {key: I_FL_arx,   value: '142.3'}
    - {key: I_FL_xgb,   value: '148.1'}
    - {key: I_FL_final, value: '148.1'} # max(arx, xgb)
    - {key: power_arx_kw, value: '71.2'}
    - {key: power_xgb_kw, value: '74.0'}
    - {key: t_arx_ms,    value: '0.0012'}
    - {key: t_xgb_ms,    value: '0.0521'}

```

Częstotliwość publikacji

```

ros2 topic hz /current_predictor/power_limit_exceeded
# average rate: 5.000

```

Tabela topiców

Topic	Kierunek	Typ	Zawartość
<code>current_sensors_data</code>	sub	Float32MultiArray	I_FL, I_FR, I_RL, I_RR [raw ADC]
<code>torque_setpoints</code>	sub	Float32MultiArray	T_FL, T_FR, T_RL, T_RR [Nm]
<code>bus_voltage</code>	sub	Float32	U_dc [V]
<code>~/predicted_currents</code>	pub	Float32MultiArray	max(ARX,XGB) per motor — do limitera
<code>~/predicted_currents_arx</code>	pub	Float32MultiArray	ARX Q=0.90 (monitorowanie)
<code>~/predicted_currents_xgb</code>	pub	Float32MultiArray	XGBoost Q=0.90 (monitorowanie)
<code>~/predicted_power_kw</code>	pub	Float32	$U_{dc} \times \Sigma I_{pred}$ [kW]
<code>~/power_limit_exceeded</code>	pub	Bool	True → ogranicz moment
<code>~/diagnostics</code>	pub	DiagnosticArray	czasy inferencji, wartości per model

Konfiguracja — params.yaml

Parametr	Domyślnie	Opis
<code>n_lags</code>	<code>5</code>	Liczba kroków historii (nie zmieniaj — musi zgadzać się z treningiem)
<code>publish_rate_hz</code>	<code>5.0</code>	Częstotliwość predykcji [Hz] — musi zgadzać się z data rate
<code>power_limit_w</code>	<code>80000.0</code>	Limit mocy [W] zgodny z regulaminem FS
<code>current_topic</code>	<code>current_sensors_data</code>	Topic z prądami inwertorów
<code>torque_topic</code>	<code>torque_setpoints</code>	Topic z setpointami momentu
<code>voltage_topic</code>	<code>bus_voltage</code>	Topic z napięciem szyny DC
<code>weights_dir</code>	(auto z package share)	Ścieżka do katalogu z wagami modeli

Troubleshooting

Node nie startuje: `FileNotFoundError: arx_q90_weights.json`

Wagi nie zostały skopiowane do pakietu. Wykonaj Kroki 1–2 i przebuduj pakiet.

```
ls ~/ros2_ws/src/current_predictor/model_weights/  
# Musi zawierać: arx_q90_weights.json, xgb_q90_FL.ubj, ...  
colcon build --packages-select current_predictor
```

Node nie publikuje danych

Bufor potrzebuje `n_lags + 1 = 6` próbek przed pierwszą predykcją (≈ 1.2 s przy 5 Hz). Sprawdź czy topic wejściowy nadaje dane:

```
ros2 topic hz /putm_vcl/current_sensor # oczekiwane: ~5 Hz
```

ModuleNotFoundError: No module named 'xgboost'

XGBoost nie jest zainstalowany w środowisku Pythona używanym przez ROS2:

```
# Znajdź Python używany przez ROS2:  
which python3 # po source /opt/ros/.../setup.bash  
  
# Zainstaluj w tym środowisku:  
python3 -m pip install xgboost
```

Predykcja nie reaguje na prądy — sprawdź kolejność silników

Kolejność w modelu (trenowanym na `training_data.csv`) to zawsze: **FL, FR, RL, RR**.

Upewnij się, że `_cb_currents` zwraca wartości w tej samej kolejności.

Reset bufora między przejazdami

Jeśli chcesz wyczyścić historię na starcie nowego przejazdu, wywołaj:

```
node._buffer.reset()
```

Lub dodaj subskrypcję na sygnał start/stop przejazdu i wywołaj `self._buffer.reset()` w callbacku.

Wygenerowano na podstawie pakietu `ros2_pkg/current_predictor` —
`PUTM_EV_FF_CurrentLoop`.